

Another common functional-style programming operation is **filtering** elements to select only those that match a condition. This typically produces a list with *fewer* elements than the data being filtered. To do this in a list comprehension, use the **if clause**. The following includes in `list4` only the even values produced by the `for` clause:

[lick here to view code image](#)

```
In [8]: list4 = [item for item in range(1, 11) if item % 2 == 0]

In [9]: list4
Out[9]: [2, 4, 6, 8, 10]
```

List Comprehension That Processes Another List's Elements

The `for` clause can process any iterable. Let's create a list of lowercase strings and use a list comprehension to create a new list containing their uppercase versions:

[lick here to view code image](#)

```
In [10]: colors = ['red', 'orange', 'yellow', 'green', 'blue']

In [11]: colors2 = [item.upper() for item in colors]

In [12]: colors2
Out[12]: ['RED', 'ORANGE', 'YELLOW', 'GREEN', 'BLUE']

In [13]: colors
Out[13]: ['red', 'orange', 'yellow', 'green', 'blue']
```

5.13 GENERATOR EXPRESSIONS

A **generator expression** is similar to a list comprehension, but creates an iterable **generator object** that produces values *on demand*. This is known as **lazy evaluation**. List comprehensions use **greedy evaluation**—they create lists *immediately* when you execute them. For large numbers of items, creating a list can take substantial memory and time. So generator expressions can reduce your program's memory consumption and improve performance if the whole list is not needed at once.

Generator expressions have the same capabilities as list comprehensions, but you define them in parentheses instead of square brackets. The generator expression in `snippet [2]` squares and returns only the odd values in `numbers`:

[lick here to view code image](#)

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [2]: for value in (x ** 2 for x in numbers if x % 2 != 0):
...:     print(value, end=' ')
...:
9 49 1 81 25
```

To show that a generator expression does not create a list, let's assign the preceding snippet's generator expression to a variable and evaluate the variable:

[lick here to view code image](#)

```
In [3]: squares_of_odds = (x ** 2 for x in numbers if x % 2 != 0)

In [3]: squares_of_odds
Out[3]: <generator object <genexpr> at 0x1085e84c0>
```

The text "generator object <genexpr>" indicates that `squares_of_odds` is a generator object that was created from a generator expression (`genexpr`).

5.14 FILTER, MAP AND REDUCE

The preceding section introduced several functional-style features—list comprehensions, filtering and mapping. Here we demonstrate the built-in `filter` and `map` functions for filtering and mapping, respectively. We continue discussing reductions in which you process a collection of elements into a *single* value, such as their count, total, product, average, minimum or maximum.

Filtering a Sequence's Values with the Built-In `filter` Function

Let's use built-in function `filter` to obtain the odd values in `numbers`:

[lick here to view code image](#)

```
In [1]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [2]: def is_odd(x):
...:     """Returns True only if x is odd."""
...:     return x % 2 != 0
...:
```

```
In [3]: list(filter(is_odd, numbers))
Out[3]: [3, 7, 1, 9, 5]
```

Like data, Python functions are objects that you can assign to variables, pass to other functions and return from functions. Functions that receive other functions as arguments are a functional-style capability called **higher-order functions**. For example, `filter`'s first argument must be a function that receives one argument and returns `True` if the value should be included in the result. The function `is_odd` returns `True` if its argument is odd. The `filter` function calls `is_odd` once for each value in its second argument's iterable (`numbers`). Higher-order functions may also return a function as a result.

Function `filter` returns an iterator, so `filter`'s results are not produced until you iterate through them. This is another example of lazy evaluation. In snippet [3], function `list` iterates through the results and creates a list containing them. We can obtain the same results as above by using a list comprehension with an `if` clause:

[lick here to view code image](#)

```
In [4]: [item for item in numbers if is_odd(item)]
Out[4]: [3, 7, 1, 9, 5]
```

Using a `lambda` Rather than a Function

For simple functions like `is_odd` that return only a *single expression's value*, you can use a **lambda expression** (or simply a **lambda**) to define the function inline where it's needed—typically as it's passed to another function:

[lick here to view code image](#)

```
In [5]: list(filter(lambda x: x % 2 != 0, numbers))
Out[5]: [3, 7, 1, 9, 5]
```

We pass `filter`'s return value (an iterator) to function `list` here to convert the results to a list and display them.

A lambda expression is an *anonymous function*—that is, a *function without a name*. In the `filter` call

[lick here to view code image](#)

```
filter(lambda x: x % 2 != 0, numbers)
```

the first argument is the lambda

```
lambda x: x % 2 != 0
```

A lambda begins with the **lambda** keyword followed by a comma-separated parameter list, a colon (:) and an expression. In this case, the parameter list has one parameter named `x`. A lambda *implicitly* returns its expression's value. So any simple function of the form

[lick here to view code image](#)

```
def function_name(parameter_list):  
    return expression
```

may be expressed as a more concise lambda of the form

[lick here to view code image](#)

```
lambda parameter_list: expression
```

Mapping a Sequence's Values to New Values

Let's use built-in function **map** with a lambda to square each value in `numbers`:

[lick here to view code image](#)

```
In [6]: numbers  
Out[6]: [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]  
  
In [7]: list(map(lambda x: x ** 2, numbers))  
Out[7]: [100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```

Function `map`'s first argument is a function that receives one value and returns a new value—in this case, a lambda that squares its argument. The second argument is an iterable of values to map. Function `map` uses lazy evaluation. So, we pass to the `list` function the iterator that `map` returns. This enables us to iterate through and create a list of the mapped values. Here's an equivalent list comprehension:

[lick here to view code image](#)

```
In [8]: [item ** 2 for item in numbers]
Out[8]: [100, 9, 49, 1, 81, 16, 4, 64, 25, 36]
```

Combining `filter` and `map`

You can combine the preceding `filter` and `map` operations as follows:

[lick here to view code image](#)

```
In [9]: list(map(lambda x: x ** 2,
...:             filter(lambda x: x % 2 != 0, numbers)))
...:
Out[9]: [9, 49, 1, 81, 25]
```

There is a lot going on in snippet [9], so let's take a closer look at it. First, `filter` returns an iterable representing only the odd values of `numbers`. Then `map` returns an iterable representing the squares of the filtered values. Finally, `list` uses `map`'s iterable to create the list. You might prefer the following list comprehension to the preceding snippet:

[lick here to view code image](#)

```
In [10]: [x ** 2 for x in numbers if x % 2 != 0]
Out[10]: [9, 49, 1, 81, 25]
```

For each value of `x` in `numbers`, the expression `x ** 2` is performed only if the condition `x % 2 != 0` is `True`.

Reduction: Totaling the Elements of a Sequence with `sum`

As you know reductions process a sequence's elements into a single value. You've performed reductions with the built-in functions `len`, `sum`, `min` and `max`. You also can create custom reductions using the `functools` module's `reduce` function. See <https://docs.python.org/3/library/functools.html> for a code example.

When we investigate big data and Hadoop in [chapter 16](#), we'll demonstrate MapReduce programming, which is based on the `filter`, `map` and `reduce` operations in functional-style programming.

5.15 OTHER SEQUENCE PROCESSING FUNCTIONS

Python provides other built-in functions for manipulating sequences.

Finding the Minimum and Maximum Values Using a Key Function

We've previously shown the built-in reduction functions `min` and `max` using arguments, such as `ints` or lists of `ints`. Sometimes you'll need to find the minimum and maximum of more complex objects, such as strings. Consider the following comparison:

```
In [1]: 'Red' < 'orange'
Out[1]: True
```

The letter 'R' “comes after” 'o' in the alphabet, so you might expect 'Red' to be less than 'orange' and the condition above to be `False`. However, strings are compared by their characters' underlying *numerical values*, and lowercase letters have *higher* numerical values than uppercase letters. You can confirm this with built-in function `ord`, which returns the numerical value of a character:

```
In [2]: ord('R')
Out[2]: 82

In [3]: ord('o')
Out[3]: 111
```

Consider the list `colors`, which contains strings with uppercase and lowercase letters:

[lick here to view code image](#)

```
In [4]: colors = ['Red', 'orange', 'Yellow', 'green', 'Blue']
```

Let's assume that we'd like to determine the minimum and maximum strings using *alphabetical* order, not *numerical* (lexicographical) order. If we arrange colors alphabetically

[lick here to view code image](#)

```
'Blue', 'green', 'orange', 'Red', 'Yellow'
```

you can see that 'Blue' is the minimum (that is, closest to the beginning of the

alphabet), and 'Yellow' is the maximum (that is, closest to the end of the alphabet).

Since Python compares strings using numerical values, you must first convert each string to all lowercase or all uppercase letters. Then their numerical values will also represent *alphabetical* ordering. The following snippets enable `min` and `max` to determine the minimum and maximum strings alphabetically:

[lick here to view code image](#)

```
In [5]: min(colors, key=lambda s: s.lower())
Out[5]: 'Blue'

In [6]: max(colors, key=lambda s: s.lower())
Out[6]: 'Yellow'
```

The `key` keyword argument must be a one-parameter function that returns a value. In this case, it's a `lambda` that calls string method `lower` to get a string's lowercase version. Functions `min` and `max` call the `key` argument's function for each element and use the results to compare the elements.

Iterating Backward Through a Sequence

Built-in function `reversed` returns an iterator that enables you to iterate over a sequence's values backward. The following list comprehension creates a new list containing the squares of `numbers`' values in reverse order:

[lick here to view code image](#)

```
In [7]: numbers = [10, 3, 7, 1, 9, 4, 2, 8, 5, 6]

In [7]: reversed_numbers = [item for item in reversed(numbers)]

In [8]: reversed_numbers
Out[8]: [36, 25, 64, 4, 16, 81, 1, 49, 9, 100]
```

Combining Iterables into Tuples of Corresponding Elements

Built-in function `zip` enables you to iterate over *multiple* iterables of data at the *same* time. The function receives as arguments any number of iterables and returns an iterator that produces tuples containing the elements at the same index in each. For example, snippet [11]'s call to `zip` produces the tuples ('Bob', 3.5), ('Sue', 4.0) and ('Amanda', 3.75) consisting of the elements at index 0, 1 and 2 of each

list, respectively:

[lick here to view code image](#)

```
In [9]: names = ['Bob', 'Sue', 'Amanda']

In [10]: grade_point_averages = [3.5, 4.0, 3.75]

In [11]: for name, gpa in zip(names, grade_point_averages):
...:     print(f'Name={name}; GPA={gpa}')
...:
Name=Bob; GPA=3.5
Name=Sue; GPA=4.0
Name=Amanda; GPA=3.75
```

We unpack each tuple into `name` and `gpa` and display them. Function `zip`'s shortest argument determines the number of tuples produced. Here both have the same length.

5.16 TWO-DIMENSIONAL LISTS

Lists can contain other lists as elements. A typical use of such nested (or multidimensional) lists is to represent **tables** of values consisting of information arranged in **rows** and **columns**. To identify a particular table element, we specify *two* indices—by convention, the first identifies the element's row, the second the element's column.

Lists that require two indices to identify an element are called **two-dimensional lists** (or **double-indexed lists** or **double-subscripted lists**). Multidimensional lists can have more than two indices. Here, we introduce two-dimensional lists.

Creating a Two-Dimensional List

Consider a two-dimensional list with three rows and four columns (i.e., a 3-by-4 list) that might represent the grades of three students who each took four exams in a course:

[lick here to view code image](#)

```
In [1]: a = [[77, 68, 86, 73], [96, 87, 89, 81], [70, 90, 86, 81]]
```

Writing the list as follows makes its row and column tabular structure clearer:

[lick here to view code image](#)

```
a = [[77, 68, 86, 73], # first student's grades
      [96, 87, 89, 81], # second student's grades
      [70, 90, 86, 81]] # third student's grades
```

Illustrating a Two-Dimensional List

The diagram below shows the list `a`, with its rows and columns of exam grade values:

	Column 0	Column 1	Column 2	Column 3
Row 0	77	68	86	73
Row 1	96	87	89	81
Row 2	70	90	86	81

Identifying the Elements in a Two-Dimensional List

The following diagram shows the names of list `a`'s elements:

	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

Diagram illustrating the naming convention for elements in a two-dimensional list `a`. The diagram shows a 3x4 grid of elements. Arrows point from the labels 'List name', 'Row index', and 'Column index' to the corresponding parts of the element name `a[2][1]` in the second row, second column.

Every element is identified by a name of the form `a[i][j]` — `a` is the list's name, and `i` and `j` are the indices that uniquely identify each element's row and column, respectively. The element names in row 0 all have 0 as the first index. The element names in column 3 all have 3 as the second index.

In the two-dimensional list `a`:

- 77, 68, 86 and 73 initialize `a[0][0]`, `a[0][1]`, `a[0][2]` and `a[0][3]`, respectively,
- 96, 87, 89 and 81 initialize `a[1][0]`, `a[1][1]`, `a[1][2]` and `a[1][3]`,

respectively, and

- 70, 90, 86 and 81 initialize `a[2][0]`, `a[2][1]`, `a[2][2]` and `a[2][3]`, respectively.

A list with m rows and n columns is called an **m-by-n list** and has $m \times n$ elements.

The following nested `for` statement outputs the rows of the preceding two-dimensional list one row at a time:

[lick here to view code image](#)

```
In [2]: for row in a:
...:     for item in row:
...:         print(item, end=' ')
...:     print()
...:
77 68 86 73
96 87 89 81
70 90 86 81
```

How the Nested Loops Execute

Let's modify the nested loop to display the list's name and the row and column indices and value of each element:

[lick here to view code image](#)

```
In [3]: for i, row in enumerate(a):
...:     for j, item in enumerate(row):
...:         print(f'a[{i}][{j}]={item}', end=' ')
...:     print()
...:
a[0][0]=77 a[0][1]=68 a[0][2]=86 a[0][3]=73
a[1][0]=96 a[1][1]=87 a[1][2]=89 a[1][3]=81
a[2][0]=70 a[2][1]=90 a[2][2]=86 a[2][3]=81
```

The outer `for` statement iterates over the two-dimensional list's rows one row at a time. During each iteration of the outer `for` statement, the inner `for` statement iterates over *each* column in the current row. So in the first iteration of the outer loop, `row 0` is

```
[77, 68, 86, 73]
```

and the nested loop iterates through this list's four elements `a[0][0]=77`, `a[0][1]=68`, `a[0][2]=86` and `a[0][3]=73`.

In the second iteration of the outer loop, `row 1` is

```
[96, 87, 89, 81]
```

and the nested loop iterates through this list's four elements `a[1][0]=96`, `a[1][1]=87`, `a[1][2]=89` and `a[1][3]=81`.

In the third iteration of the outer loop, `row 2` is

```
[70, 90, 86, 81]
```

and the nested loop iterates through this list's four elements `a[2][0]=70`, `a[2][1]=90`, `a[2][2]=86` and `a[2][3]=81`.

In the “Array-Oriented Programming with NumPy” chapter, we’ll cover the NumPy library’s `ndarray` collection and the Pandas library’s `DataFrame` collection. These enable you to manipulate multidimensional collections more concisely and conveniently than the two-dimensional list manipulations you’ve seen in this section.

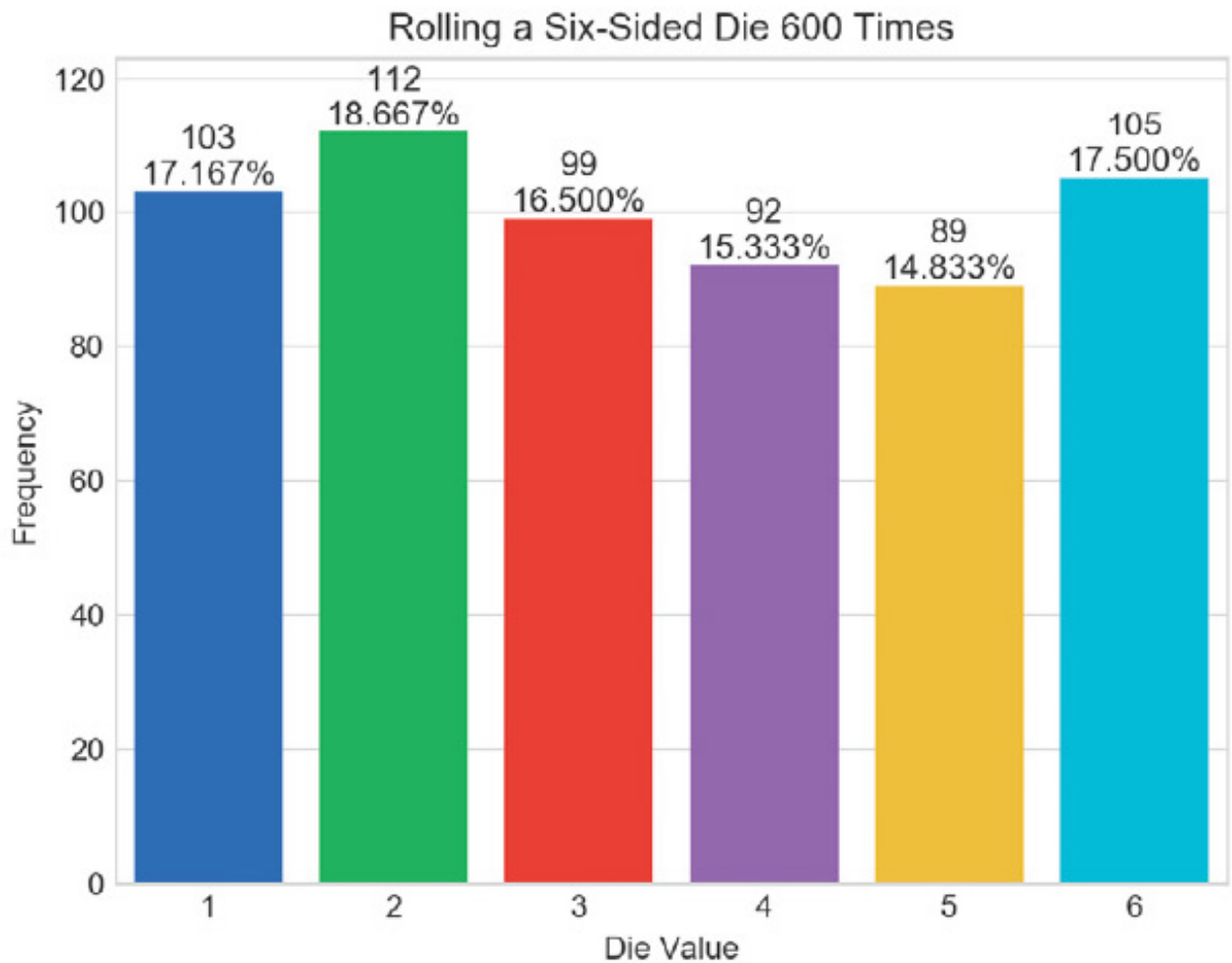
5.17 INTRO TO DATA SCIENCE: SIMULATION AND STATIC VISUALIZATIONS

The last few chapters’ Intro to Data Science sections discussed basic descriptive statistics. Here, we focus on visualizations, which help you “get to know” your data. Visualizations give you a powerful way to understand data that goes beyond simply looking at raw data.

We use two open-source visualization libraries—Seaborn and Matplotlib—to display *static* bar charts showing the final results of a six-sided-die-rolling simulation. The **Seaborn visualization library** is built over the **Matplotlib visualization library** and simplifies many Matplotlib operations. We’ll use aspects of both libraries, because some of the Seaborn operations return objects from the Matplotlib library. In the next chapter’s Intro to Data Science section, we’ll make things “come alive” with *dynamic visualizations*.

5.17.1 Sample Graphs for 600, 60,000 and 6,000,000 Die Rolls

he screen capture below shows a vertical bar chart that for 600 die rolls summarizes the frequencies with which each of the six faces appear, and their percentages of the total. Seaborn refers to this type of graph as a **bar plot**:



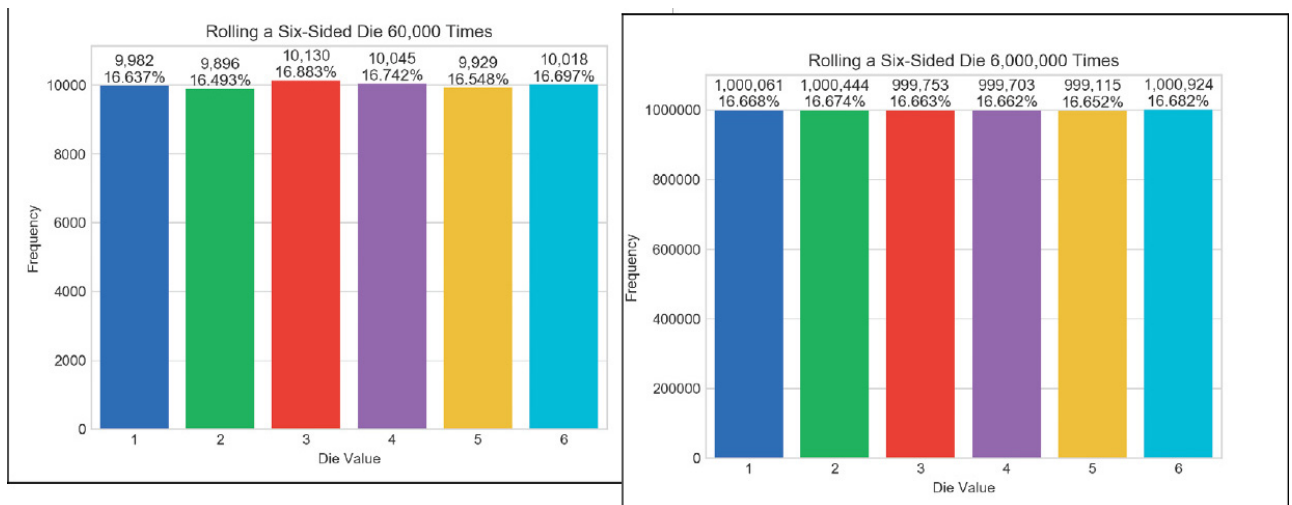
Here we expect about 100 occurrences of each die face. However, with such a small number of rolls, none of the frequencies is exactly 100 (though several are close) and most of the percentages are not close to 16.667% (about 1/6th). As we run the simulation for 60,000 die rolls, the bars will become much closer in size. At 6,000,000 die rolls, they'll appear to be exactly the same size. This is the “law of large numbers” at work. The next chapter will show the lengths of the bars changing dynamically.

We'll discuss how to control the plot's appearance and contents, including:

- the graph title inside the window (**Rolling a Six-Sided Die 600 Times**),
- the descriptive labels **Die Value** for the x-axis and **Frequency** for the y-axis,
- the text displayed above each bar, representing the *frequency* and *percentage* of the total rolls, and
- the bar colors.

We'll use various Seaborn default options. For example, Seaborn determines the text labels along the x-axis from the die face values 1–6 and the text labels along the *y*-axis from the actual die frequencies. Behind the scenes, Matplotlib determines the positions and sizes of the bars, based on the window size and the magnitudes of the values the bars represent. It also positions the **Frequency** axis's numeric labels based on the actual die frequencies that the bars represent. There are many more features you can customize. You should tweak these attributes to your personal preferences.

The first screen capture below shows the results for 60,000 die rolls—imagine trying to do this by hand. In this case, we expect about 10,000 of each face. The second screen capture below shows the results for 6,000,000 rolls—surely something you'd never do by hand! In this case, we expect about 1,000,000 of each face, and the frequency bars appear to be identical in length (they're close but not exactly the same length). Note that with more die rolls, the frequency percentages are much closer to the expected 16.667%.



5.17.2 Visualizing Die-Roll Frequencies and Percentages

In this section, you'll interactively develop the bar plots shown in the preceding section.

Launching IPython for Interactive Matplotlib Development

IPython has built-in support for interactively developing Matplotlib graphs, which you also need to develop Seaborn graphs. Simply launch IPython with the command:

```
ipython --matplotlib
```

Importing the Libraries

First, let's import the libraries we'll use:

[lick here to view code image](#)

```
In [1]: import matplotlib.pyplot as plt

In [2]: import numpy as np

In [3]: import random

In [4]: import seaborn as sns
```

1. The **matplotlib.pyplot module** contains the Matplotlib library's graphing capabilities that we use. This module typically is imported with the name `plt`.
2. The NumPy (Numerical Python) library includes the function `unique` that we'll use to summarize the die rolls. The **numpy module** typically is imported as `np`.
3. The `random` module contains Python's random-number-generation functions.
4. The **seaborn module** contains the Seaborn library's graphing capabilities we use. This module typically is imported with the name `sns`. Search for why this curious abbreviation was chosen.

Rolling the Die and Calculating Die Frequencies

Next, let's use a *list comprehension* to create a list of 600 random die values, then use NumPy's **unique** function to determine the unique roll values (most likely all six possible face values) and their frequencies:

[lick here to view code image](#)

```
In [5]: rolls = [random.randrange(1, 7) for i in range(600)]

In [6]: values, frequencies = np.unique(rolls, return_counts=True)
```

The NumPy library provides the high-performance **ndarray** collection, which is typically much faster than lists.¹ Though we do not use `ndarray` directly here, the NumPy `unique` function expects an `ndarray` argument and returns an `ndarray`. If you pass a list (like `rolls`), NumPy converts it to an `ndarray` for better performance. The `ndarray` that `unique` returns we'll simply assign to a variable for use by a Seaborn plotting function.

¹ We'll run a performance comparison in [chapter 7](#) where we discuss `ndarray` in

depth.

Specifying the keyword argument `return_counts=True` tells `unique` to count each unique value's number of occurrences. In this case, `unique` returns a tuple of two one-dimensional `ndarrays` containing the sorted unique values and the corresponding frequencies, respectively. We unpack the tuple's `ndarrays` into the variables `values` and `frequencies`. If `return_counts` is `False`, only the list of unique values is returned.

Creating the Initial Bar Plot

Let's create the bar plot's title, set its style, then graph the die faces and frequencies:

[lick here to view code image](#)

```
In [7]: title = f'Rolling a Six-Sided Die {len(rolls):,} Times'

In [8]: sns.set_style('whitegrid')

In [9]: axes = sns.barplot(x=values, y=frequencies, palette='bright')
```

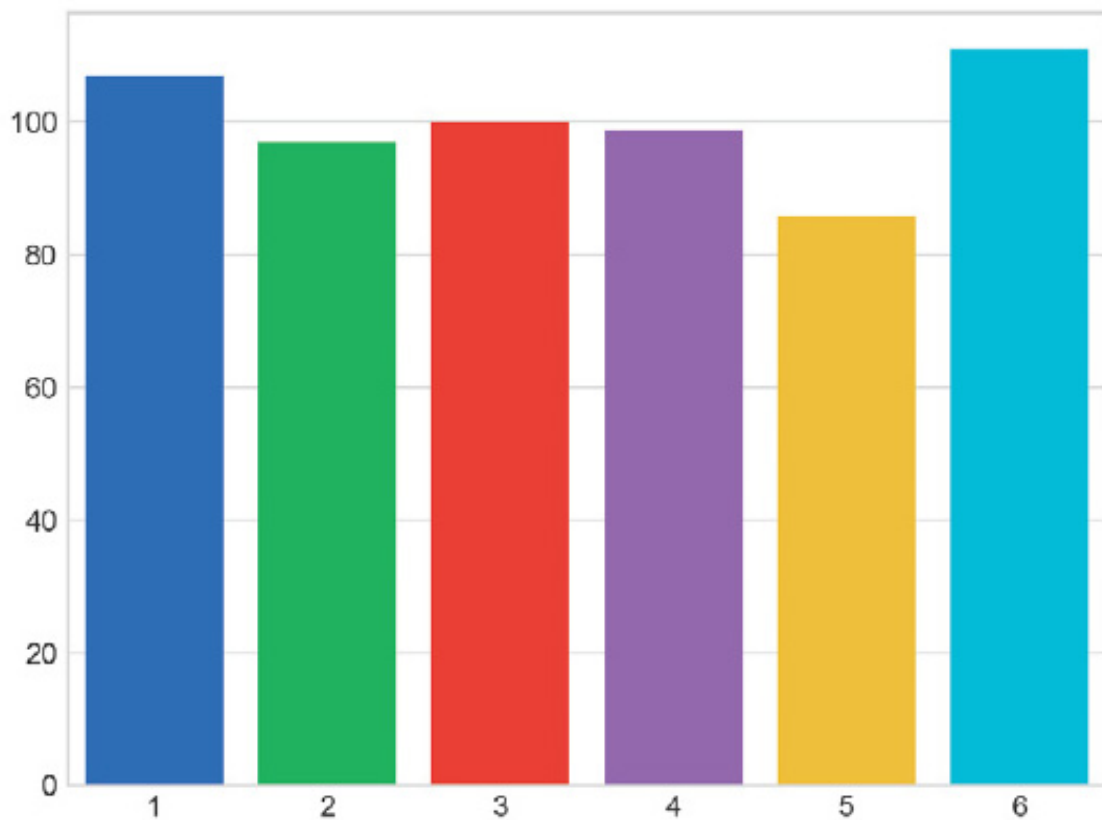
Snippet [7]'s f-string includes the number of die rolls in the bar plot's title. The comma (,) format specifier in

```
{len(rolls):,}
```

displays the number with *thousands separators*—so, 60000 would be displayed as 60,000.

By default, Seaborn plots graphs on a plain white background, but it provides several styles to choose from ('darkgrid', 'whitegrid', 'dark', 'white' and 'ticks'). Snippet [8] specifies the 'whitegrid' style, which displays light-gray horizontal lines in the vertical bar plot. These help you see more easily how each bar's height corresponds to the numeric frequency labels at the bar plot's left side.

Snippet [9] graphs the die frequencies using Seaborn's **barplot function**. When you execute this snippet, the following window appears (because you launched IPython with the `--matplotlib` option):



Seaborn interacts with Matplotlib to display the bars by creating a Matplotlib **Axes** object, which manages the content that appears in the window. Behind the scenes, Seaborn uses a Matplotlib **Figure** object to manage the window in which the Axes will appear. Function `barplot`'s first two arguments are `ndarrays` containing the *x*-axis and *y*-axis values, respectively. We used the optional `palette` keyword argument to choose Seaborn's predefined color palette 'bright'. You can view the palette options at:

https://seaborn.pydata.org/tutorial/color_palettes.html

Function `barplot` returns the Axes object that it configured. We assign this to the variable `axes` so we can use it to configure other aspects of our final plot. Any changes you make to the bar plot after this point will appear *immediately* when you execute the corresponding snippet.

Setting the Window Title and Labeling the x- and y-Axes

The next two snippets add some descriptive text to the bar plot:

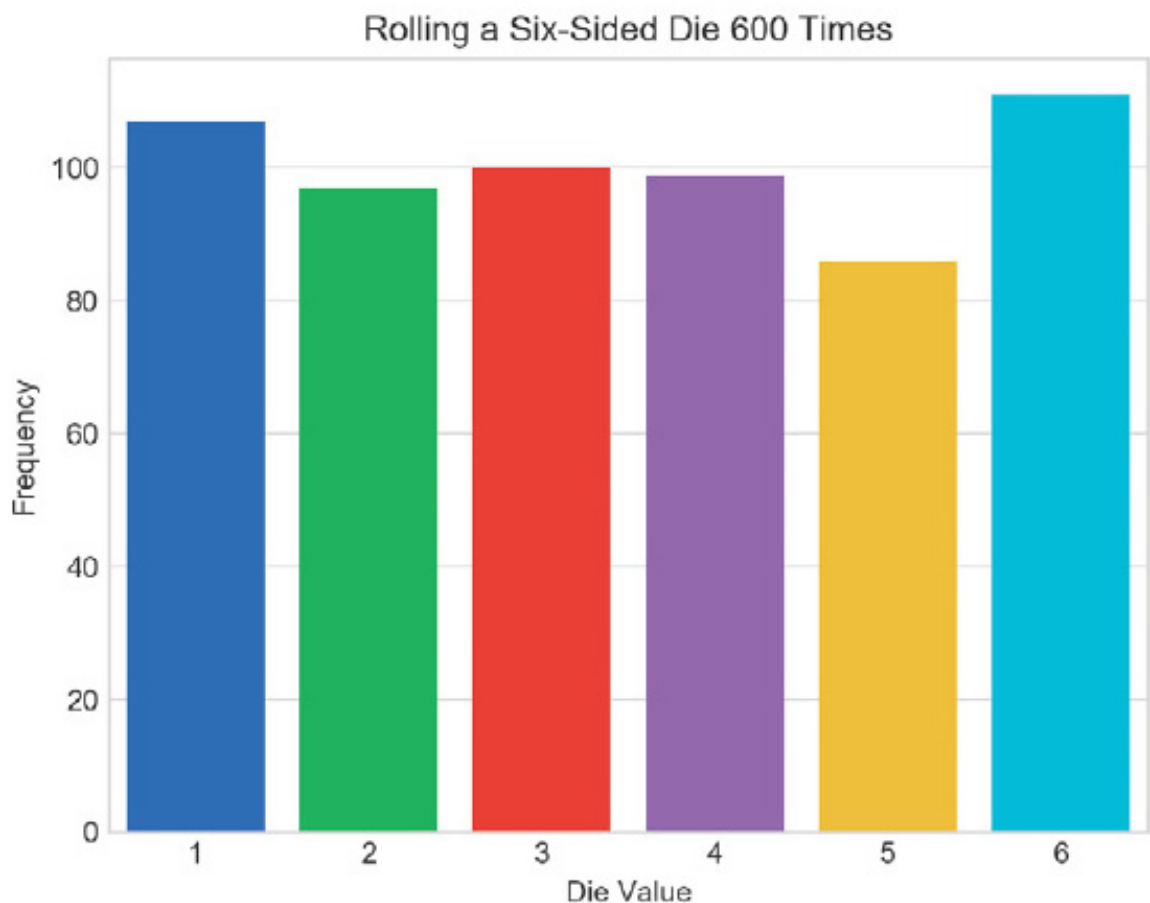
[lick here to view code image](#)

```
In [10]: axes.set_title(title)
Out[10]: Text(0.5,1,'Rolling a Six-Sided Die 600 Times')
```

```
In [11]: axes.set(xlabel='Die Value',    ylabel='Frequency')
Out[11]: [Text(92.6667,0.5,'Frequency'), Text(0.5,58.7667,'Die    Value')]
```

Snippet [10] uses the `axes` object's `set_title` method to display the `title` string centered above the plot. This method returns a `Text` object containing the title and its *location* in the window, which IPython simply displays as output for confirmation. You can ignore the `Out[]`s in the snippets above.

Snippet [11] add labels to each axis. The `set` method receives keyword arguments for the `Axes` object's properties to set. The method displays the `xlabel` text along the *x*-axis, and the `ylabel`- text along the *y*-axis, and returns a list of `Text` objects containing the labels and their locations. The bar plot now appears as follows:



Finalizing the Bar Plot

The next two snippets complete the graph by making room for the text above each bar, then displaying it:

[lick here to view code image](#)

```
In [12]: axes.set_ylim(top=max(frequencies) * 1.10)
```

```

Out[12]: (0.0, 122.10000000000001)

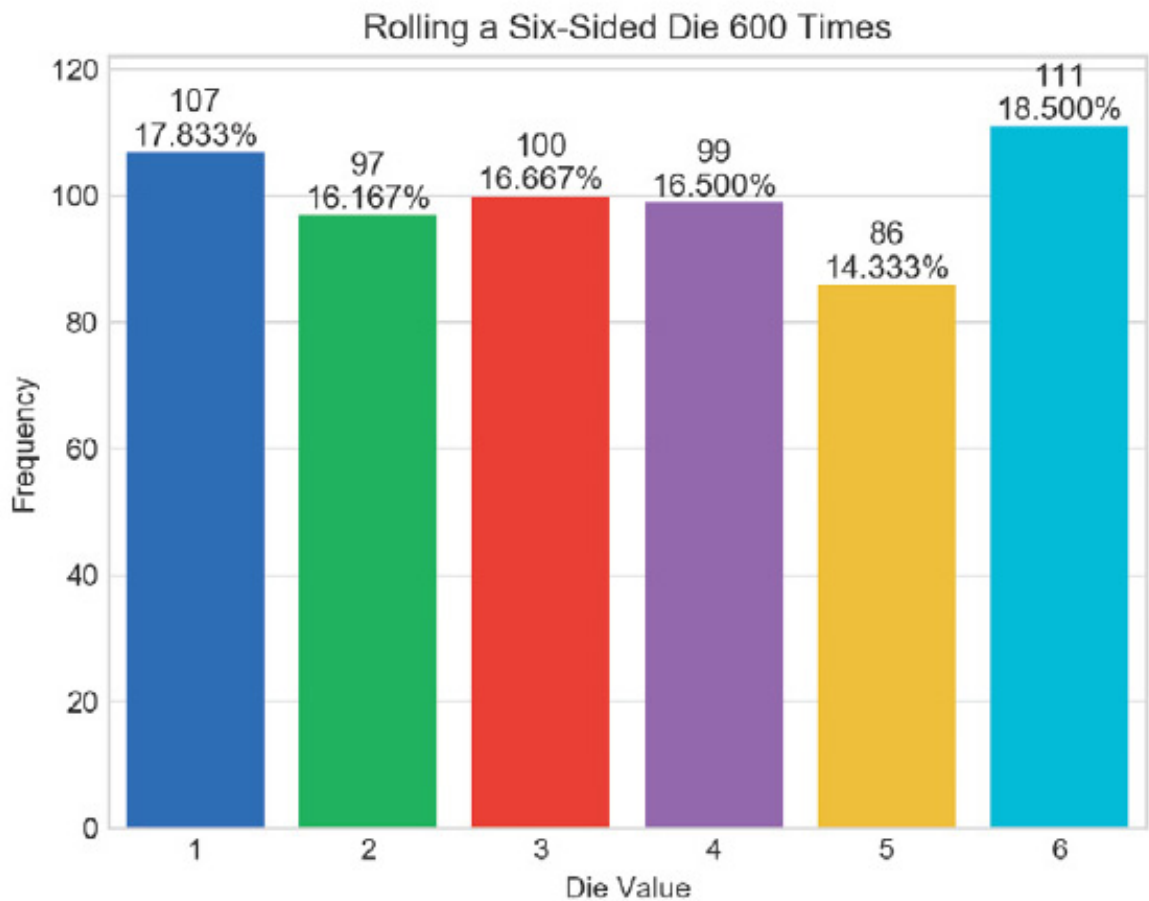
In [13]: for bar, frequency in zip(axes.patches, frequencies):
...:     text_x = bar.get_x() + bar.get_width() / 2.0
...:     text_y = bar.get_height()
...:     text = f'{frequency:},\n{frequency / len(rolls):.3%}'
...:     axes.text(text_x, text_y, text,
...:                fontsize=11, ha='center', va='bottom')
...:

```

To make room for the text above the bars, snippet [12] scales the y -axis by 10%. We chose this value via experimentation. The `Axes` object's `set_ylim` method has many optional keyword arguments. Here, we use only `top` to change the maximum value represented by the y -axis. We multiplied the largest frequency by 1.10 to ensure that the y -axis is 10% taller than the tallest bar.

Finally, snippet [13] displays each bar's frequency value and percentage of the total rolls. The `axes` object's `patches` collection contains two-dimensional colored shapes that represent the plot's bars. The `for` statement uses `zip` to iterate through the `patches` and their corresponding frequency values. Each iteration unpacks into `bar` and `frequency` one of the tuples `zip` returns. The `for` statement's suite operates as follows:

- The first statement calculates the center x -coordinate where the text will appear. We calculate this as the sum of the bar's left-edge x -coordinate (`bar.get_x()`) and half of the bar's width (`bar.get_width() / 2.0`).
- The second statement gets the y -coordinate where the text will appear—`bar.get_y()` represents the bar's top.
- The third statement creates a two-line string containing that bar's frequency and the corresponding percentage of the total die rolls.
- The last statement calls the `Axes` object's `text` method to display the text above the bar. This method's first two arguments specify the text's x - y position, and the third argument is the text to display. The keyword argument `ha` specifies the *horizontal alignment*—we centered text horizontally around the x -coordinate. The keyword argument `va` specifies the *vertical alignment*—we aligned the bottom of the text with at the y -coordinate. The final bar plot is shown below:



Rolling Again and Updating the Bar Plot—Introducing IPython Magics

Now that you’ve created a nice bar plot, you probably want to try a different number of die rolls. First, clear the existing graph by calling Matplotlib’s `cla` (clear axes) function:

```
In [14]: plt.cla()
```

IPython provides special commands called **magics** for conveniently performing various tasks. Let’s use the **%recall magic** to get snippet [5], which created the `rolls` list, and place the code at the next `In []` prompt:

[lick here to view code image](#)

```
In [15]: %recall 5
```

```
In [16]: rolls = [random.randrange(1, 7) for i in range(600)]
```

You can now edit the snippet to change the number of rolls to 60000, then press *Enter* to create a new list:

[lick here to view code image](#)

```
In [16]: rolls = [random.randrange(1, 7) for i in range(60000)]
```

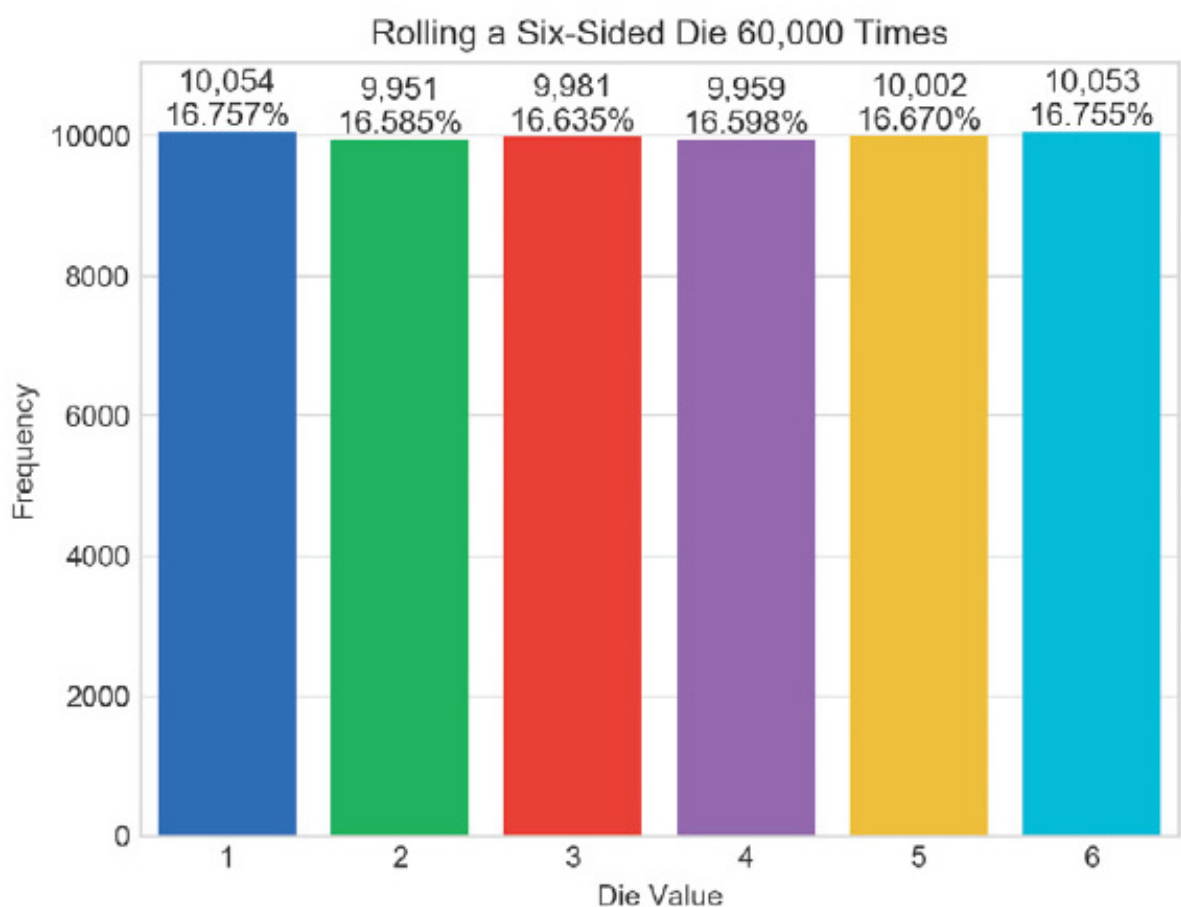
Next, recall snippets [6] through [13]. This displays all the snippets in the specified range in the next In [] prompt. Press *Enter* to re-execute these snippets:

[lick here to view code image](#)

```
In [17]: %recall 6-13
```

```
In [18]: values, frequencies = np.unique(rolls, return_counts=True)
...: title = f'Rolling a Six-Sided Die {len(rolls):,} Times'
...: sns.set_style('whitegrid')
...: axes = sns.barplot(x=values, y=frequencies, palette='bright')
...: axes.set_title(title)
...: axes.set_xlabel('Die Value', ylabel='Frequency')
...: axes.set_ylim(top=max(frequencies) * 1.10)
...: for bar, frequency in zip(axes.patches, frequencies):
...:     text_x = bar.get_x() + bar.get_width() / 2.0
...:     text_y = bar.get_height()
...:     text = f'{frequency:,}\n{frequency / len(rolls):.3%}'
...:     axes.text(text_x, text_y, text,
...:                 fontsize=11, ha='center', va='bottom')
...:
```

The updated bar plot is shown below:



Saving Snippets to a File with the `%save` Magic

Once you've interactively created a plot, you may want to save the code to a file so you can turn it into a script and run it in the future. Let's use the `%save magic` to save snippets 1 through 13 to a file named `RollDie.py`. IPython indicates the file to which the lines were written, then displays the lines that it saved:

[lick here to view code image](#)

```
In [19]: %save RollDie.py 1-13
The following commands were written to file `RollDie.py`:
import matplotlib.pyplot as plt
import numpy as np
import random
import seaborn as sns
rolls = [random.randrange(1, 7) for i in range(600)]
values, frequencies = np.unique(rolls, return_counts=True)
title = f'Rolling a Six-Sided Die {len(rolls):,} Times'
sns.set_style("whitegrid")
axes = sns.barplot(values, frequencies, palette='bright')
axes.set_title(title)
axes.set_xlabel('Die Value', ylabel='Frequency')
axes.set_ylim(top=max(frequencies) * 1.10)
for bar, frequency in zip(axes.patches, frequencies):
    text_x = bar.get_x() + bar.get_width() / 2.0
    text_y = bar.get_height()
    text = f'{frequency:,}\n{frequency / len(rolls):.3%}'
    axes.text(text_x, text_y, text,
              fontsize=11, ha='center', va='bottom')
```

Command-Line Arguments; Displaying a Plot from a Script

Provided with this chapter's examples is an edited version of the `RollDie.py` file you saved above. We added comments and a two modifications so you can run the script with an argument that specifies the number of die rolls, as in:

```
ipython RollDie.py 600
```

The Python Standard Library's `sys module` enables a script to receive *command-line arguments* that are passed into the program. These include the script's name and any values that appear to the right of it when you execute the script. The `sys` module's `argv` list contains the arguments. In the command above, `argv[0]` is the *string* `'RollDie.py'` and `argv[1]` is the *string* `'600'`. To control the number of die rolls with the command-line argument's value, we modified the statement that creates the `rolls` list as follows:

[lick here to view code image](#)

```
rolls = [random.randrange(1, 7) for i in range(int(sys.argv[1]))]
```

Note that we converted the `argv[1]` string to an `int`.

Matplotlib and Seaborn do not automatically display the plot for you when you create it in a script. So at the end of the script we added the following call to Matplotlib's `show` function, which displays the window containing the graph:

```
plt.show()
```

5.18 WRAP-UP

This chapter presented more details of the list and tuple sequences. You created lists, accessed their elements and determined their length. You saw that lists are mutable, so you can modify their contents, including growing and shrinking the lists as your programs execute. You saw that accessing a nonexistent element causes an `IndexError`. You used `for` statements to iterate through list elements.

We discussed tuples, which like lists are sequences, but are immutable. You unpacked a tuple's elements into separate variables. You used `enumerate` to create an iterable of tuples, each with a list index and corresponding element value.

You learned that all sequences support slicing, which creates new sequences with subsets of the original elements. You used the `del` statement to remove elements from lists and delete variables from interactive sessions. We passed lists, list elements and slices of lists to functions. You saw how to search and sort lists, and how to search tuples. We used list methods to insert, append and remove elements, and to reverse a list's elements and copy lists.

We showed how to simulate stacks with lists. We used the concise list-comprehension notation to create new lists. We used additional built-in methods to sum list elements, iterate backward through a list, find the minimum and maximum values, filter values and map values to new values. We showed how nested lists can represent two-dimensional tables in which data is arranged in rows and columns. You saw how nested `for` loops process two-dimensional lists.

The chapter concluded with an Intro to Data Science section that presented a die-

olling simulation and static visualizations. A detailed code example used the Seaborn and Matplotlib visualization libraries to create a *static* bar plot visualization of the simulation’s final results. In the next Intro to Data Science section, we use a die-rolling simulation with a *dynamic* bar plot visualization to make the plot “come alive.”

In the next chapter, “Dictionaries and Sets,” we’ll continue our discussion of Python’s built-in collections. We’ll use dictionaries to store unordered collections of key–value pairs that map immutable keys to values, just as a conventional dictionary maps words to definitions. We’ll use sets to store unordered collections of unique elements.

In the “Array-Oriented Programming with NumPy” chapter, we’ll discuss NumPy’s `ndarray` collection in more detail. You’ll see that while lists are fine for small amounts of data, they are not efficient for the large amounts of data you’ll encounter in big data analytics applications. For such cases, the NumPy library’s highly optimized `ndarray` collection should be used. `ndarray` (*n*-dimensional array) can be much faster than lists. We’ll run Python profiling tests to see just how much faster. As you’ll see, NumPy also includes many capabilities for conveniently and efficiently manipulating arrays of *many* dimensions. In big data analytics applications, the processing demands can be humongous, so everything we can do to improve performance significantly matters. In our “Big Data: Hadoop, Spark, NoSQL and IoT” chapter, you’ll use one of the most popular high-performance big-data databases—MongoDB.²

² The databases name is rooted in the word humongous.

6. Dictionaries and Sets

Objectives

In this chapter, you'll:

- Use dictionaries to represent unordered collections of key–value pairs.
- Use sets to represent unordered collections of unique values.
- Create, initialize and refer to elements of dictionaries and sets.
- Iterate through a dictionary's keys, values and key–value pairs.
- Add, remove and update a dictionary's key–value pairs.
- Use dictionary and set comparison operators.
- Combine sets with set operators and methods.
- Use operators `in` and `not in` to determine if a dictionary contains a key or a set contains a value.
- Use the mutable set operations to modify a set's contents.
- Use comprehensions to create dictionaries and sets quickly and conveniently.
- Learn how to build dynamic visualizations.
- Enhance your understanding of mutability and immutability.

Outline

.1 Introduction

.2 Dictionaries

.2.1 Creating a Dictionary

.2.2 Iterating through a Dictionary

.2.3 Basic Dictionary Operations

.2.4 Dictionary Methods `keys` and `values`

.2.5 Dictionary Comparisons

.2.6 Example: Dictionary of Student Grades

.2.7 Example: Word Counts

.2.8 Dictionary Method `update`

.2.9 Dictionary Comprehensions

.3 Sets

.3.1 Comparing Sets

.3.2 Mathematical Set Operations

.3.3 Mutable Set Operators and Methods

.3.4 Set Comprehensions

.4 Intro to Data Science: Dynamic Visualizations

.4.1 How Dynamic Visualization Works

.4.2 Implementing a Dynamic Visualization

.5 Wrap-Up

6.1 INTRODUCTION

We've discussed three built-in sequence collections—strings, lists and tuples. Now, we consider the built-in non-sequence collections—dictionaries and sets. A **dictionary** is an *unordered* collection which stores **key–value pairs** that map immutable keys to values, just as a conventional dictionary maps words to definitions. A **set** is an

unordered collection of *unique* immutable elements.

6.2 DICTIONARIES

A dictionary *associates* keys with values. Each key *maps* to a specific value. The following table contains examples of dictionaries with their keys, key types, values and value types:

Keys	Key type	Values	Value type
Country names	<code>str</code>	Internet country codes	<code>str</code>
Decimal numbers	<code>int</code>	Roman numerals	<code>str</code>
States	<code>str</code>	Agricultural products	list of <code>str</code>
Hospital patients	<code>str</code>	Vital signs	tuple of <code>ints</code> and <code>floats</code>
Baseball players	<code>str</code>	Batting averages	<code>float</code>
Metric measurements	<code>str</code>	Abbreviations	<code>str</code>
Inventory codes	<code>str</code>	Quantity in stock	<code>int</code>

unique Keys

A dictionary's keys must be *immutable* (such as strings, numbers or tuples) and *unique* (that is, no duplicates). Multiple keys can have the same value, such as two different inventory codes that have the same quantity in stock.

6.2.1 Creating a Dictionary

You can create a dictionary by enclosing in curly braces, {}, a comma-separated list of key–value pairs, each of the form *key*: *value*. You can create an empty dictionary with {}.

Let's create a dictionary with the country-name keys 'Finland', 'South Africa' and 'Nepal' and their corresponding Internet country code values 'fi', 'za' and 'np':

[lick here to view code image](#)

```
In [1]: country_codes = {'Finland': 'fi', 'South Africa': 'za',
...:                    'Nepal': 'np'}
...:

In [2]: country_codes
Out[2]: {'Finland': 'fi', 'South Africa': 'za', 'Nepal': 'np'}
```

When you output a dictionary, its comma-separated list of key–value pairs is always enclosed in curly braces. Because dictionaries are *unordered* collections, the display order can differ from the order in which the key–value pairs were added to the dictionary. In snippet [2]'s output the key–value pairs are displayed in the order they were inserted, but do *not* write code that depends on the order of the key–value pairs.

Determining if a Dictionary Is Empty

The built-in function `len` returns the number of key–value pairs in a dictionary:

```
In [3]: len(country_codes)
Out[3]: 3
```

You can use a dictionary as a condition to determine if it's empty—a non-empty dictionary evaluates to `True`:

[lick here to view code image](#)

```
In [4]: if country_codes:
...:     print('country_codes is not empty')
...: else:
...:     print('country_codes is empty')
...:
country_codes is not empty
```

An empty dictionary evaluates to `False`. To demonstrate this, in the following code we call method `clear` to delete the dictionary's key-value pairs, then in snippet [6] we recall and re-execute snippet [4]:

[lick here to view code image](#)

```
In [5]: country_codes.clear()

In [6]: if country_codes:
...:     print('country_codes is not empty')
...: else:
...:     print('country_codes is empty')
...:
country_codes is empty
```

6.2.2 Iterating through a Dictionary

The following dictionary maps month-name strings to `int` values representing the numbers of days in the corresponding month. Note that *multiple* keys can have the *same* value:

[lick here to view code image](#)

```
In [1]: days_per_month = {'January': 31, 'February': 28, 'March': 31}

In [2]: days_per_month
Out[2]: {'January': 31, 'February': 28, 'March': 31}
```

Again, the dictionary's string representation shows the key-value pairs in their insertion order, but this is not guaranteed because dictionaries are *unordered*. We'll show how to process keys in *sorted* order later in this chapter.

The following `for` statement iterates through `days_per_month`'s key-value pairs. Dictionary method `items` returns each key-value pair as a tuple, which we unpack into `month` and `days`:

[lick here to view code image](#)

```
In [3]: for month, days in days_per_month.items():
...:     print(f'{month} has {days} days')
...:
January has 31 days
February has 28 days
March has 31 days
```

6.2.3 Basic Dictionary Operations

For this section, let's begin by creating and displaying the dictionary `roman_numerals`. We intentionally provide the incorrect value 100 for the key 'X', which we'll correct shortly:

[lick here to view code image](#)

```
In [1]: roman_numerals = {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}

In [2]: roman_numerals
Out[2]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}
```

Accessing the Value Associated with a Key

Let's get the value associated with the key 'V':

```
In [3]: roman_numerals['V']
Out[3]: 5
```

Updating the Value of an Existing Key-Value Pair

You can update a key's associated value in an assignment statement, which we do here to replace the incorrect value associated with the key 'X':

[lick here to view code image](#)

```
In [4]: roman_numerals['X'] = 10

In [5]: roman_numerals
Out[5]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10}
```

Adding a New Key-Value Pair

Assigning a value to a nonexistent key inserts the key–value pair in the dictionary:

[lick here to view code image](#)

```
In [6]: roman_numerals['L'] = 50

In [7]: roman_numerals
Out[7]: {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10, 'L': 50}
```

String keys are case sensitive. Assigning to a nonexistent key inserts a new key–value pair. This may be what you intend, or it could be a logic error.

Removing a Key-Value Pair

You can delete a key–value pair from a dictionary with the `del` statement:

[lick here to view code image](#)

```
In [8]: del roman_numerals['III']

In [9]: roman_numerals
Out[9]: {'I': 1, 'II': 2, 'V': 5, 'X': 10, 'L': 50}
```

You also can remove a key–value pair with the dictionary method `pop`, which returns the value for the removed key:

[lick here to view code image](#)

```
In [10]: roman_numerals.pop('X')
Out[10]: 10

In [11]: roman_numerals
Out[11]: {'I': 1, 'II': 2, 'V': 5, 'L': 50}
```

Attempting to Access a Nonexistent Key

Accessing a nonexistent key results in a `KeyError`:

[lick here to view code image](#)

```
n [12]: roman_numerals['III']
```

KeyError Traceback (most recent call last)

```
<ipython-input-12-ccd50c7f0c8b> in <module>()
----> 1 roman_numerals['III']
```

```
KeyError: 'III'
```

ou can prevent this error by using dictionary method **get**, which normally returns its argument's corresponding value. If that key is not found, **get** returns `None`. IPython does not display anything when `None` is returned in snippet [13]. If you specify a second argument to **get**, it returns that value if the key is not found:

[lick here to view code image](#)

```
In [13]: roman_numerals.get('III')

In [14]: roman_numerals.get('III', 'III not in dictionary')
Out[14]: 'III not in dictionary'

In [15]: roman_numerals.get('V')
Out[15]: 5
```

Testing Whether a Dictionary Contains a Specified Key

Operators `in` and `not in` can determine whether a dictionary contains a specified key:

[lick here to view code image](#)

```
In [16]: 'V' in roman_numerals
Out[16]: True

In [17]: 'III' in roman_numerals
Out[17]: False

In [18]: 'III' not in roman_numerals
Out[18]: True
```

6.2.4 Dictionary Methods **keys** and **values**

Earlier, we used dictionary method `items` to iterate through tuples of a dictionary's key–value pairs. Similarly, methods **keys** and **values** can be used to iterate through only a dictionary's keys or values, respectively:

[lick here to view code image](#)

```
In [1]: months = {'January': 1, 'February': 2, 'March': 3}

In [2]: for month_name in months.keys():
...:     print(month_name, end=' ')
...:
January February March

In [3]: for month_number in months.values():
...:     print(month_number, end=' ')
...:
1 2 3
```

Dictionary Views

Dictionary methods `items`, `keys` and `values` each return a view of a dictionary's data. When you iterate over a **view**, it “sees” the dictionary's current contents—it does *not* have its own copy of the data.

To show that views do *not* maintain their own copies of a dictionary's data, let's first save the view returned by `keys` into the variable `months_view`, then iterate through it:

[lick here to view code image](#)

```
In [4]: months_view = months.keys()

In [5]: for key in months_view:
...:     print(key, end=' ')
...:
January February March
```

Next, let's add a new key–value pair to `months` and display the updated dictionary:

[lick here to view code image](#)

```
In [6]: months['December'] = 12

In [7]: months
Out[7]: {'January': 1, 'February': 2, 'March': 3, 'December': 12}
```

Now, let's iterate through `months_view` again. The key we added above is indeed displayed:

[lick here to view code image](#)

```
In [8]: for key in months_view:
...:     print(key, end=' ')
...:
January February March December
```

Do not modify a dictionary while iterating through a view. According to Section 4.10.1 of the Python Standard Library documentation,¹ either you'll get a `RuntimeError` or the loop might not process all of the view's values.

¹ <https://docs.python.org/3/library/stdtypes.html#dictionary-view-objects>.

Converting Dictionary Keys, Values and Key-Value Pairs to Lists

You might occasionally need *lists* of a dictionary's keys, values or key-value pairs. To obtain such a list, pass the view returned by `keys`, `values` or `items` to the built-in `list` function. Modifying these lists does *not* modify the corresponding dictionary:

[lick here to view code image](#)

```
In [9]: list(months.keys())
Out[9]: ['January', 'February', 'March', 'December']

In [10]: list(months.values())
Out[10]: [1, 2, 3, 12]

In [11]: list(months.items())
Out[11]: [('January', 1), ('February', 2), ('March', 3), ('December', 12)]
```

rocessing Keys in Sorted Order

To process keys in *sorted* order, you can use built-in function `sorted` as follows:

[lick here to view code image](#)

```
In [12]: for month_name in sorted(months.keys()):
...:     print(month_name, end=' ')
...:
February December January March
```

6.2.5 Dictionary Comparisons

The comparison operators `==` and `!=` can be used to determine whether two dictionaries have identical or different contents. An equals (`==`) comparison evaluates to `True` if both dictionaries have the same key–value pairs, *regardless* of the order in which those key–value pairs were added to each dictionary:

[lick here to view code image](#)

```
In [1]: country_capitals1 = {'Belgium': 'Brussels',
...:                        'Haiti': 'Port-au-Prince'}
...:

In [2]: country_capitals2 = {'Nepal': 'Kathmandu',
...:                         'Uruguay': 'Montevideo'}
...:

In [3]: country_capitals3 = {'Haiti': 'Port-au-Prince',
...:                         'Belgium': 'Brussels'}
...:

In [4]: country_capitals1 == country_capitals2
Out[4]: False

In [5]: country_capitals1 == country_capitals3
Out[5]: True

In [6]: country_capitals1 != country_capitals2
Out[6]: True
```

6.2.6 Example: Dictionary of Student Grades

The following script represents an instructor’s grade book as a dictionary that maps each student’s name (a string) to a list of integers containing that student’s grades on three exams. In each iteration of the loop that displays the data (lines 13–17), we unpack a key–value pair into the variables `name` and `grades` containing one student’s name and the corresponding list of three grades. Line 14 uses built-in function `sum` to total a given student’s grades, then line 15 calculates and displays that student’s average by dividing `total` by the number of grades for that student (`len(grades)`). Lines 16–17 keep track of the total of all four students’ grades and the number of grades for all the students, respectively. Line 19 prints the class average of all the students’ grades on all the exams.

[lick here to view code image](#)

```
1 # fig06_01.py
2 """Using a dictionary to represent an instructor's grade book."""
```

```

3 grade_book = {
4     'Susan': [92, 85, 100],
5     'Eduardo': [83, 95, 79],
6     'Azizi': [91, 89, 82],
7     'Pantipa': [97, 91, 92]
8 }
9
10 all_grades_total = 0
11 all_grades_count = 0
12
13 for name, grades in grade_book.items():
14     total = sum(grades)
15     print(f'Average for {name} is {total/len(grades):.2f}')
16     all_grades_total += total
17     all_grades_count += len(grades)
18
19 print(f"Class's average is: {all_grades_total / all_grades_count:.2f}")

```

[lick here to view code image](#)

```

Average for Susan is 92.33
Average for Eduardo is 85.67
Average for Azizi is 87.33
Average for Pantipa is 93.33
Class's average is: 89.67

```

6.2.7 Example: Word Counts ²

² Techniques like word frequency counting are often used to analyze published works. For example, some people believe that the works of William Shakespeare actually might have been written by Sir Francis Bacon, Christopher Marlowe or others. Comparing the word frequencies of their works with those of Shakespeare can reveal writing-style similarities. We'll look at other document-analysis techniques in the Natural Language Processing (NLP) chapter.

The following script builds a dictionary to count the number of occurrences of each word in a string. Lines 4–5 create a string `text` that we'll break into words—a process known as **tokenizing a string**. Python automatically concatenates strings separated by whitespace in parentheses. Line 7 creates an empty dictionary. The dictionary's keys will be the unique words, and its values will be integer counts of how many times each word appears in `text`.

[lick here to view code image](#)

```
1 # fig06_02.py
2 """Tokenizing a string and counting unique words."""
3
4 text = ('this is sample text with several words '
5         'this is more sample text with some different words')
6
7 word_counts = {}
8
9 # count occurrences of each unique word
10 for word in text.split():
11     if word in word_counts:
12         word_counts[word] += 1 # update existing key-value pair
13     else:
14         word_counts[word] = 1 # insert new key-value pair
15
16 print(f'{"WORD":<12}COUNT')
17
18 for word, count in sorted(word_counts.items()):
19     print(f'{word:<12}{count}')
20
21 print('\nNumber of unique words:', len(word_counts))
```

[lick here to view code image](#)

WORD	COUNT
different	1
is	2
more	1
sample	2
several	1
some	1
text	2
this	2
with	2
words	2
Number of unique words: 10	

Line 10 tokenizes `text` by calling string method `split`, which separates the words using the method's delimiter string argument. If you do not provide an argument, `split` uses a space. The method returns a list of tokens (that is, the words in `text`). Lines 10–14 iterate through the list of words. For each `word`, line 11 determines whether that `word` (the key) is already in the dictionary. If so, line 12 increments that word's count; otherwise, line 14 inserts a new key–value pair for that `word` with an

initial count of 1.

Lines 16–21 summarize the results in a two-column table containing each `word` and its corresponding `count`. The `for` statement in lines 18 and 19 iterates through the dictionary's key–value pairs. It unpacks each key and value into the variables `word` and `count`, then displays them in two columns. Line 21 displays the number of unique words.

Python Standard Library Module `collections`

The Python Standard Library already contains the counting functionality that we implemented using the dictionary and the loop in lines 10–14. The module `collections` contains the type `Counter`, which receives an iterable and summarizes its elements. Let's reimplement the preceding script in fewer lines of code with `Counter`:

[lick here to view code image](#)

```
In [1]: from collections import Counter

In [2]: text = ('this is sample text    with several words '
...:           'this is more sample    text with some different words')
...:

In [3]: counter = Counter(text.split())

In [4]: for word, count in sorted(counter.items()):
...:     print(f'{word:<12}{count}')
...:
different      1
is              2
more           1
sample         2
several        1
some           1
text           2
this           2
with           2
words          2

In [5]: print('Number of unique    keys:', len(counter.keys()))
Number of unique keys: 10
```

Snippet [3] creates the `Counter`, which summarizes the list of strings returned by `text.split()`. In snippet [4], `Counter` method `items` returns each string and its associated count as a tuple. We use built-in function `sorted` to get a list of these tuples

in ascending order. By default sorted orders the tuples by their first elements. If those are identical, then it looks at the second element, and so on. The `for` statement iterates over the resulting sorted list, displaying each `word` and `count` in two columns.

6.2.8 Dictionary Method `update`

You may insert and update key–value pairs using dictionary method `update`. First, let's create an empty `country_codes` dictionary:

```
In [1]: country_codes = {}
```

The following `update` call receives a dictionary of key–value pairs to insert or update:

[lick here to view code image](#)

```
In [2]: country_codes.update({'South Africa': 'za'})

In [3]: country_codes
Out[3]: {'South Africa': 'za'}
```

Method `update` can convert keyword arguments into key–value pairs to insert. The following call automatically converts the parameter name `Australia` into the string key `'Australia'` and associates the value `'ar'` with that key:

[lick here to view code image](#)

```
In [4]: country_codes.update(Australia='ar')

In [5]: country_codes
Out[5]: {'South Africa': 'za', 'Australia': 'ar'}
```

Snippet [4] provided an incorrect country code for Australia. Let's correct this by using another keyword argument to update the value associated with `'Australia'`:

[lick here to view code image](#)

```
In [6]: country_codes.update(Australia='au')

In [7]: country_codes
Out[7]: {'South Africa': 'za', 'Australia': 'au'}
```

Method `update` also can receive an iterable object containing key–value pairs, such as a list of two-element tuples.

6.2.9 Dictionary Comprehensions

Dictionary comprehensions provide a convenient notation for quickly generating dictionaries, often by mapping one dictionary to another. For example, in a dictionary with *unique* values, you can reverse the key–value pairs:

[lick here to view code image](#)

```
In [1]: months = {'January': 1, 'February': 2, 'March': 3}

In [2]: months2 = {number: name for name, number in months.items()}

In [3]: months2
Out[3]: {1: 'January', 2: 'February', 3: 'March'}
```

Curly braces delimit a *dictionary comprehension*, and the expression to the left of the `for` clause specifies a key–value pair of the form *key*: *value*. The comprehension iterates through `months.items()`, unpacking each key–value pair tuple into the variables `name` and `number`. The expression `number: name` reverses the key and value, so the new dictionary maps the month numbers to the month names.

What if `months` contained *duplicate* values? As these become the keys in `months2`, attempting to insert a *duplicate* key simply updates the existing key’s value. So if `'February'` and `'March'` both mapped to 2 originally, the preceding code would have produced

```
{1: 'January', 2: 'March'}
```

A dictionary comprehension also can map a dictionary’s values to new values. The following comprehension converts a dictionary of names and lists of grades into a dictionary of names and grade-point averages. The variables `k` and `v` commonly mean *key* and *value*:

[lick here to view code image](#)

```
In [4]: grades = {'Sue': [98, 87, 94], 'Bob': [84, 95, 91]}

In [5]: grades2 = {k: sum(v) / len(v) for k, v in grades.items()}
```

```
In [6]: grades2
Out[6]: {'Sue': 93.0, 'Bob': 90.0}
```

The comprehension unpacks each tuple returned by `grades.items()` into `k` (the name) and `v` (the list of grades). Then, the comprehension creates a new key–value pair with the key `k` and the value of `sum(v) / len(v)`, which averages the list’s elements.

6.3 SETS

A set is an unordered collection of *unique* values. Sets may contain only immutable objects, like strings, ints, floats and tuples that contain only immutable elements. Though sets are iterable, they are not sequences and do not support indexing and slicing with square brackets, `[]`. Dictionaries also do not support slicing.

Creating a Set with Curly Braces

The following code creates a set of strings named `colors`:

[lick here to view code image](#)

```
In [1]: colors = {'red', 'orange', 'yellow', 'green', 'red', 'blue'}

In [2]: colors
Out[2]: {'blue', 'green', 'orange', 'red', 'yellow'}
```

Notice that the duplicate string `'red'` was ignored (without causing an error). An important use of sets is **duplicate elimination**, which is automatic when creating a set. Also, the resulting set’s values are *not* displayed in the same order as they were listed in snippet [1]. Though the color names are displayed in sorted order, sets are *unordered*. You should not write code that depends on the order of their elements.

Determining a Set’s Length

You can determine the number of items in a set with the built-in `len` function:

```
In [3]: len(colors)
Out[3]: 5
```

Checking Whether a Value Is in a Set

You can check whether a set contains a particular value using the `in` and `not in`

operators:

[lick here to view code image](#)

```
In [4]: 'red' in colors
Out[4]: True

In [5]: 'purple' in colors
Out[5]: False

In [6]: 'purple' not in colors
Out[6]: True
```

Iterating Through a Set

Sets are iterable, so you can process each set element with a `for` loop:

[lick here to view code image](#)

```
In [7]: for color in colors:
...:     print(color.upper(), end=' ')
...:
RED GREEN YELLOW BLUE ORANGE
```

Sets are *unordered*, so there's no significance to the iteration order.

Creating a Set with the Built-In `set` Function

You can create a set from another collection of values by using the built-in `set` function—here we create a list that contains several duplicate integer values and use that list as `set`'s argument:

[lick here to view code image](#)

```
In [8]: numbers = list(range(10)) + list(range(5))

In [9]: numbers
Out[9]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4]

In [10]: set(numbers)
Out[10]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

If you need to create an empty set, you must use the `set` function with empty parentheses, rather than empty braces, `{ }`, which represent an empty dictionary:

```
In [11]: set()
Out[11]: set()
```

Python displays an empty set as `set()` to avoid confusion with Python's string representation of an empty dictionary (`{}`).

Frozenset: An Immutable Set Type

Sets are *mutable*—you can add and remove elements, but set *elements* must be *immutable*. Therefore, a set cannot have other sets as elements. A **frozenset** is an *immutable* set—it cannot be modified after you create it, so a set *can* contain frozensets as elements. The built-in function **frozenset** creates a frozenset from any iterable.

6.3.1 Comparing Sets

Various operators and methods can be used to compare sets. The following sets contain the same values, so `==` returns `True` and `!=` returns `False`.

[lick here to view code image](#)

```
In [1]: {1, 3, 5} == {3, 5, 1}
Out[1]: True

In [2]: {1, 3, 5} != {3, 5, 1}
Out[2]: False
```

The `<` operator tests whether the set to its left is a **proper subset** of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets are not equal:

[lick here to view code image](#)

```
In [3]: {1, 3, 5} < {3, 5, 1}
Out[3]: False

In [4]: {1, 3, 5} < {7, 3, 5, 1}
Out[4]: True
```

The `<=` operator tests whether the set to its left is an **improper subset** of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets might be equal:

[lick here to view code image](#)

```
In [5]: {1, 3, 5} <= {3, 5, 1}
Out[5]: True

In [6]: {1, 3} <= {3, 5, 1}
Out[6]: True
```

You may also check for an improper subset with the set method **issubset**:

[lick here to view code image](#)

```
In [7]: {1, 3, 5}.issubset({3, 5, 1})
Out[7]: True

In [8]: {1, 2}.issubset({3, 5, 1})
Out[8]: False
```

The **>** operator tests whether the set to its left is a **proper superset** of the one to its right—that is, all the elements in the right operand are in the left operand, and the left operand has more elements:

[lick here to view code image](#)

```
In [9]: {1, 3, 5} > {3, 5, 1}
Out[9]: False

In [10]: {1, 3, 5, 7} > {3, 5, 1}
Out[10]: True
```

The **>=** operator tests whether the set to its left is an **improper superset** of the one to its right—that is, all the elements in the right operand are in the left operand, and the sets might be equal:

[lick here to view code image](#)

```
In [11]: {1, 3, 5} >= {3, 5, 1}
Out[11]: True

In [12]: {1, 3, 5} >= {3, 1}
Out[12]: True

In [13]: {1, 3} >= {3, 1, 7}
Out[13]: False
```

You may also check for an improper superset with the set method **issuperset**:

[lick here to view code image](#)

```
In [14]: {1, 3, 5}.issuperset({3, 5, 1})
Out[14]: True

In [15]: {1, 3, 5}.issuperset({3, 2})
Out[15]: False
```

The argument to `issubset` or `issuperset` can be *any* iterable. When either of these methods receives a non-set iterable argument, it first converts the iterable to a set, then performs the operation.

6.3.2 Mathematical Set Operations

This section presents the set type's mathematical operators `|`, `&`, `-` and `^` and the corresponding methods.

Union

The **union** of two sets is a set consisting of all the unique elements from both sets. You can calculate the union with the **| operator** or with the set type's **union** method:

[lick here to view code image](#)

```
In [1]: {1, 3, 5} | {2, 3, 4}
Out[1]: {1, 2, 3, 4, 5}

In [2]: {1, 3, 5}.union([20, 20, 3, 40, 40])
Out[2]: {1, 3, 5, 20, 40}
```

The operands of the binary set operators, like `|`, must both be sets. The corresponding set methods may receive any iterable object as an argument—we passed a list. When a mathematical set method receives a non-set iterable argument, it first converts the iterable to a set, then applies the mathematical operation. Again, though the new sets' string representations show the values in ascending order, you should not write code that depends on this.

Intersection

The **intersection** of two sets is a set consisting of all the unique elements that the two

sets have in common. You can calculate the intersection with the **& operator** or with the set type's **intersection** method:

[lick here to view code image](#)

```
In [3]: {1, 3, 5} & {2, 3, 4}
Out[3]: {3}

In [4]: {1, 3, 5}.intersection([1, 2, 2, 3, 3, 4, 4])
Out[4]: {1, 3}
```

Difference

The **difference** between two sets is a set consisting of the elements in the left operand that are not in the right operand. You can calculate the difference with the **- operator** or with the set type's **difference** method:

[lick here to view code image](#)

```
In [5]: {1, 3, 5} - {2, 3, 4}
Out[5]: {1, 5}

In [6]: {1, 3, 5, 7}.difference([2, 2, 3, 3, 4, 4])
Out[6]: {1, 5, 7}
```

Symmetric Difference

The **symmetric difference** between two sets is a set consisting of the elements of both sets that are not in common with one another. You can calculate the symmetric difference with the **^ operator** or with the set type's **symmetric_difference** method:

[lick here to view code image](#)

```
In [7]: {1, 3, 5} ^ {2, 3, 4}
Out[7]: {1, 2, 4, 5}

In [8]: {1, 3, 5, 7}.symmetric_difference([2, 2, 3, 3, 4, 4])
Out[8]: {1, 2, 4, 5, 7}
```

Disjoint

Two sets are **disjoint** if they do not have any common elements. You can determine

this with the set type's **isdisjoint** method:

[lick here to view code image](#)

```
In [9]: {1, 3, 5}.isdisjoint({2, 4, 6})
Out[9]: True

In [10]: {1, 3, 5}.isdisjoint({4, 6, 1})
Out[10]: False
```

6.3.3 Mutable Set Operators and Methods

The operators and methods presented in the preceding section each result in a *new* set. Here we discuss operators and methods that modify an *existing* set.

Mutable Mathematical Set Operations

Like operator `|`, **union augmented assignment** `|=` performs a set union operation, but `|=` modifies its left operand:

[lick here to view code image](#)

```
In [1]: numbers = {1, 3, 5}

In [2]: numbers |= {2, 3, 4}

In [3]: numbers
Out[3]: {1, 2, 3, 4, 5}
```

Similarly, the set type's **update** method performs a union operation modifying the set on which it's called—the argument can be any iterable:

[lick here to view code image](#)

```
In [4]: numbers.update(range(10))

In [5]: numbers
Out[5]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

The other mutable set methods are:

- intersection augmented assignment `&=`

- difference augmented assignment `-=`
- symmetric difference augmented assignment `^=`

and their corresponding methods with iterable arguments are:

- `intersection_update`
- `difference_update`
- `symmetric_difference_update`

Methods for Adding and Removing Elements

Set method **`add`** inserts its argument if the argument is *not* already in the set; otherwise, the set remains unchanged:

[lick here to view code image](#)

```
In [6]: numbers.add(17)

In [7]: numbers.add(3)

In [8]: numbers
Out[8]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 17}
```

Set method **`remove`** removes its argument from the set—a `KeyError` occurs if the value is not in the set:

[lick here to view code image](#)

```
In [9]: numbers.remove(3)

In [10]: numbers
Out[10]: {0, 1, 2, 4, 5, 6, 7, 8, 9, 17}
```

Method **`discard`** also removes its argument from the set but does not cause an exception if the value is not in the set.

You also can remove an *arbitrary* set element and return it with **`pop`**, but sets are unordered, so you do not know which element will be returned:

[lick here to view code image](#)

```
In [11]: numbers.pop()
Out[11]: 0

In [12]: numbers
Out[12]: {1, 2, 4, 5, 6, 7, 8, 9, 17}
```

A `KeyError` occurs if the set is empty when you call `pop`.

Finally, method `clear` empties the set on which it's called:

```
In [13]: numbers.clear()

In [14]: numbers
Out[14]: set()
```

6.3.4 Set Comprehensions

Like dictionary comprehensions, you define set comprehensions in curly braces. Let's create a new set containing only the unique even values in the list `numbers`:

[lick here to view code image](#)

```
In [1]: numbers = [1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10, 10]

In [2]: evens = {item for item in numbers if item % 2 == 0}

In [3]: evens
Out[3]: {2, 4, 6, 8, 10}
```

6.4 INTRO TO DATA SCIENCE: DYNAMIC VISUALIZATIONS

The preceding chapter's Intro to Data Science section introduced visualization. We simulated rolling a six-sided die and used the Seaborn and Matplotlib visualization libraries to create a publication-quality *static* bar plot showing the frequencies and percentages of each roll value. In this section, we make things “come alive” with *dynamic visualizations*.

The Law of Large Numbers

hen we introduced random-number generation, we mentioned that if the `random` module's `randrange` function indeed produces integers at random, then every number in the specified range has an equal probability (or likelihood) of being chosen each time the function is called. For a six-sided die, each value 1 through 6 should occur one-sixth of the time, so the probability of any one of these values occurring is $1/6^{\text{th}}$ or about 16.667%.

In the next section, we create and execute a *dynamic* (that is, *animated*) die-rolling simulation script. In general, you'll see that the more rolls we attempt, the closer each die value's percentage of the total rolls gets to 16.667% and the heights of the bars gradually become about the same. This is a manifestation of the *law of large numbers*.

6.4.1 How Dynamic Visualization Works

The plots produced with Seaborn and Matplotlib in the previous chapter's Intro to Data Science section help you analyze the results for a fixed number of die rolls *after* the simulation completes. This section's enhances that code with the Matplotlib **animation** module's **FuncAnimation** function, which updates the bar plot *dynamically*. You'll see the bars, die frequencies and percentages “come alive,” updating *continuously* as the rolls occur.

Animation Frames

`FuncAnimation` drives a **frame-by-frame animation**. Each **animation frame** specifies everything that should change during one plot update. Stringing together many of these updates over time creates the animation effect. You decide what each frame displays with a function you define and pass to `FuncAnimation`.

Each animation frame will:

- roll the dice a specified number of times (from 1 to as many as you'd like), updating die frequencies with each roll,
- clear the current plot,
- create a new set of bars representing the updated frequencies, and
- create new frequency and percentage text for each bar.

Generally, displaying more frames-per-second yields smoother animation. For example, video games with fast-moving elements try to display *at least* 30 frames-per-

second and often more. Though you'll specify the number of milliseconds between animation frames, the actual number of frames-per-second can be affected by the amount of work you perform in each frame and the speed of your computer's processor. This example displays an animation frame every 33 milliseconds—yielding approximately 30 ($1000 / 33$) frames-per-second. Try larger and smaller values to see how they affect the animation. Experimentation is important in developing the best visualizations.

Running `RollDieDynamic.py`

In the previous chapter's Intro to Data Science section, we developed the static visualization *interactively* so you could see how the code updates the bar plot as you execute each statement. The actual bar plot with the final frequencies and percentages was drawn only once.

For this dynamic visualization, the screen results update frequently so that you can see the animation. Many things change continuously—the lengths of the bars, the frequencies and percentages above the bars, the spacing and labels on the axes and the total number of die rolls shown in the plot's title. For this reason, we present this visualization as a script, rather than interactively developing it.

The script takes two command-line arguments:

- `number_of_frames`—The number of animation frames to display. This value determines the total number of times that `FuncAnimation` updates the graph. For each animation frame, `FuncAnimation` calls a function that you define (in this example, `update`) to specify how to change the plot.
- `rolls_per_frame`—The number of times to roll the die in each animation frame. We'll use a loop to roll the die this number of times, summarize the results, then update the graph with bars and text representing the new frequencies.

To understand how we use these two values, consider the following command:

```
ipython RollDieDynamic.py 6000 1
```

In this case, `FuncAnimation` calls our `update` function 6000 times, rolling one die per frame for a total of 6000 rolls. This enables you to see the bars, frequencies and percentages update one roll at a time. On our system, this animation took about 3.33 minutes ($6000 \text{ frames} / 30 \text{ frames-per-second} / 60 \text{ seconds-per-minute}$) to show you